

Lab 06 Thinking in Objects

Objective

In this lab, you will learn how to:

- Declaring classes
- Dealing with Rational Numbers
- Creating objects from classes
- Declaring methods to deal with data fields of the class
- Dealing with array of objects

Activity Description

Design a class called `RelationalNumber` that represents fractions of the form (a/b) numerator/denominator where a and b are integers, (3/4, 1/3, 1/2. etc.).

Declare class **Rational Number** with the following attributes:

1. Declare the numerator and the denominator as private data fields.
2. Define a constructor to initialize the data fields. The header as follows:
public RationalNumber (int numer, int denom);
3. Implement all the needed methods to perform the addition and multiplication operations as follows.
 - ❖ $a/b + c/d = (ad + bc)/bd$
 - ❖ $a/b \times c/d = ac/bd$
4. Implement the `toString()` method to print object data.

Hint: the addition and multiplication methods parameter and return type will be of type `RationalNumber`. The headers for them are:

```
public RationalNumber add (RationalNumber op2);  
public RationalNumber multiply (RationalNumber op2);
```

Implement class **RationalArray** that contains the main class:

- Store the data in **List1** and **List2** according to the below table.

List 1	List 2	List1 + List2	List1*List2
3/4	2/5	23/20	6/20
1/3	1/4	7/12	1/12
1/2	3/5	11/10	3/10

- Perform the addition and multiplication operations on the given corresponding items in the lists, and then display the four Lists.
- Declare 4 arrays of objects of type RationalNumber where :
 - list1: to store 3 rational numbers**
 - list2: to store another 3 rational numbers**
 - List3: stores (List1 + List2)**
 - List4: stores (List1*List2) .**
- Fill list1 and list2 arrays with data from the table.
- Call the addition and multiplication methods to fill List1 + List2 and List1*List2.
- Print the result: print the content of List1 + List2 and List1*List2

Sample Output:

```
run:
LIST1    LIST2    ADD      MULTIPLY
3/4      2/5      23/20    6/20
1/3      1/4      7/12     1/12
1/2      3/5      11/10    3/10
BUILD SUCCESSFUL (total time: 0 seconds)
```